

ФКН ВШЭ
Глубинное обучение 1, 2024/25

Автор конспекта: Даниил Тихонов

27 марта 2025 г. в 14:21

Содержание

1. Соревновательные атаки, генеративно-сопязательные сети (GANs)	1
1.1. Соревновательные атаки (Adversarial attacks)	1
1.2. Генеративно-сопязательные сети (GANs)	3
1.3. Различные улучшения	4
1.4. Теоретические обоснование	6
1.5. Mode collapse	7

Лекция #13: Соревновательные атаки, генеративно-сопоставительные сети (GANs)

3 февраля, 2025

1.1. Соревновательные атаки (Adversarial attacks)

Пусть имеется объект (x, y_{true}) , обученная нейросеть $f_{\theta}(x)$ с параметрами θ и некоторая функция потерь $\mathcal{L}(\theta, x, y)$. Предположим, что злоумышленник хочет испортить нашу модель, но он не имеет прямого доступа к самой модели (не может менять ее веса). Единственный способ ее испортить – манипулировать входом x .

Более формальная формулировка следующая. Предполагая, что модель обучена хорошо, то есть $f_{\theta}(x) = y_{true}$, нужно найти такой $\hat{x} \approx x$, чтобы модель ошибалась: $f_{\theta}(\hat{x}) \neq y_{true}$.

В целом возникает вопрос об устойчивости моделей к такого рода атакам. В идеале мы бы хотели, чтобы предсказания моделей не менялись от добавления ко входу маленького шума.

Формально, назовем ε **бюджетом атаки**, где

$$\|x - \hat{x}\|_{\infty} = \max_i |x_i - \hat{x}_i| < \frac{\varepsilon}{255}$$

Будем специально искать объекты, которые удовлетворяют бюджету атаки и на которых модель ошибается больше всего. Сначала введем классификацию атак:

- **White Box** атаки – мы имеем доступ к модели и ее весам, а значит можем считать градиент.
- **Black Box** атаки – нет доступа к модели, а следовательно и ее градиентам.

Второй случай сильно сложнее первого и разбираться подробно не будет. Один из возможных способов строить Black Box атаки: взять статью про желаемую модель, попытаться обучить ее самому и надеяться, что градиенты обученной модели будут похожи на градиенты недоступной модели.

Например, это можно сделать путем скрейпинга выходов модели, лежащей где-то на сервере, и дообучении на этот выход. Таким образом, мы приближаем нашу обученную модель к исходной, ожидая сходство градиентов.

White Box атаки тоже можно разделить на два класса:

- Таргетированные (**Targeted attack**). Мы хотим, чтобы модель ошибалась определенным образом, а именно выдавала заранее выбранный нами класс: $f_{\theta}(\hat{x}) = y_{target}$.
- Нетаргетированные (**Untargeted attack**). Мы просто хотим, чтобы модель ошибалась, при этом итоговое предсказание модели не очень важно: $f_{\theta}(\hat{x}) \neq y_{true}$.

С методами все получается довольно просто. В случае White Box атак у нас есть доступ к градиенту, а значит мы можем использовать его для манипуляции уверенностью модели.

Fast Gradient Sign Method (FGSM)

- Untargeted attack. Посчитаем градиент по **входу** и сделаем шаг градиентного **подъема**:

$$\hat{x} = x + \varepsilon \operatorname{sign} \left[\nabla_x \mathcal{L}(\theta, x, y_{true}) \right]$$

Интересный факт про torch. Может показаться, что `eval` режим и `torch.no_grad()` имеют одинаковый эффект на модель. Однако в первом случае есть возможность считать градиенты по входу, а во втором никакие градиенты считать нельзя.

Использование именно знака градиента связано с несколькими вещами:

- Во-первых, если взять просто градиент, то он с большой вероятностью имеет огромные значения для какого-то небольшого числа пикселей (см. хитмапы активаций сверточных сетей), а значит мы можем очень быстро за бюджет атаки.
- Во-вторых, для большинства пикселей будет околонулевой градиент, а значит, сделав обычный шаг, значения соответствующих пикселей почти не изменятся.

Когда мы берем знак, градиент портится, и мы идем не совсем в том направлении, куда нужно. Но зато все пиксели одновременно меняются на одинаковое маленькое значение.

- Targeted attack. Посчитаем градиент по **входу** и сделаем шаг градиентного **спуска**:

$$\hat{x} = x - \varepsilon \operatorname{sign} \left[\nabla_x \mathcal{L}(\theta, x, y_{target}) \right]$$

Мотивация аналогична, просто в этом случае делаем шаг в сторону желаемого класса.

Iterative FGSM (iFGSM)

Естественно возникает вопрос, почему бы не сделать несколько шагов, вместо одного.

Пусть $\hat{x}_0 := x$. Далее N раз повторяем следующий шаг:

$$\hat{x}_i := \hat{x}_{i-1} + \frac{\varepsilon}{N} \left[\nabla_x \mathcal{L}(\theta, \hat{x}_{i-1}, y_{true}) \right]$$

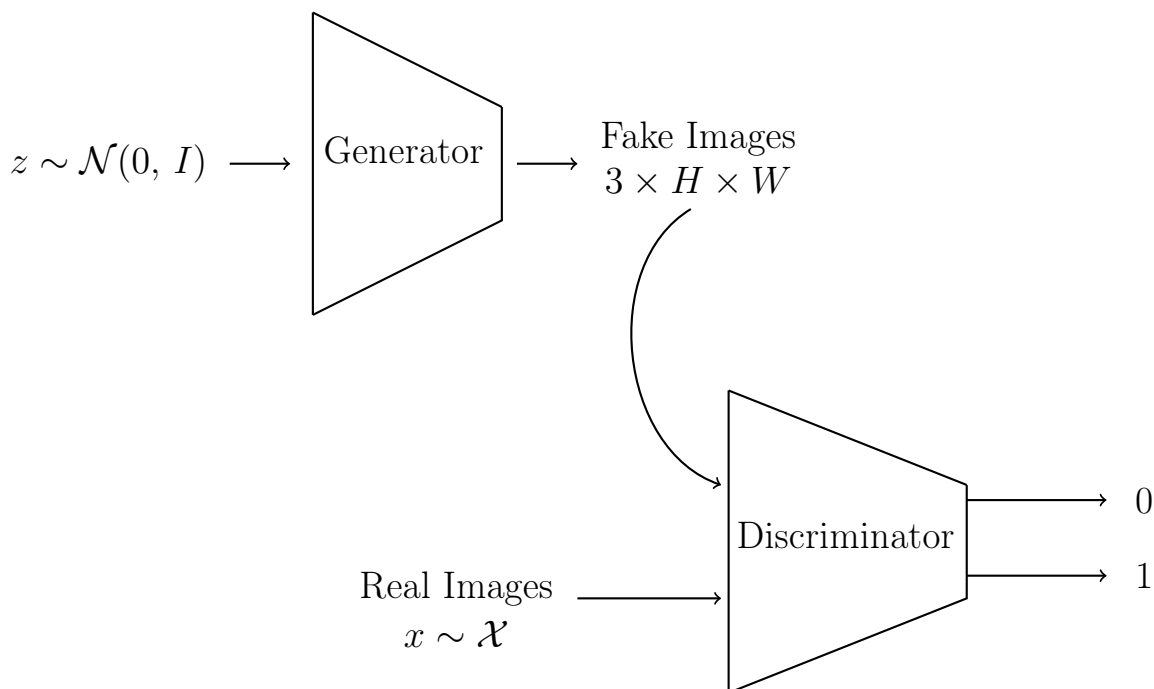
В качестве входа используется $\hat{x}_N$. Аналогично для untargeted случая. Итеративные атаки куда более эффективные: по сравнению с обычным FGSM за гораздо меньший бюджет можно добиваться очень хороших результатов.

1.2. Генеративно-состязательные сети (GANs)

Идея очень похожа на соревновательные атаки, где мы двигали предсказание в нужную нам сторону, чтобы обмануть модель. Попробуем использовать это для генерации.

Для этого нам потребуются две модели – генератор и дискриминатор.

- **Generator** получает на вход *латентную переменную* $z \sim \mathcal{N}(0, I)$. Далее модель каким-то образом обрабатывает вектор z и из него генерирует фейковые картинки.
- **Discriminator** получает на вход настоящие и фейковые картинки из генератора, а затем учится их отличать, решая задачу бинарной классификации.



Обучаться генератор и дискриминатор будут по очереди. Дискриминатор на своем шаге будет учиться как можно лучше отличать настоящие картинки от фейковых, а генератор будет пытаться его обмануть с помощью механизмов состязательной атаки: генерировать картинки, которые дискриминатор примет за реальные.

Процедура обучения зачастую очень нестабильна, так как модели не помогают друг другу, а соревнуются (отсюда и название GAN). Завести это все дело – нетривиальная задача.

Стоит также заметить, что в этой архитектуре генератор вообще не видит картинки из реального датасета: весь сигнал для обучения приходит исключительно от дискриминатора. Другими словами все упирается в качество дискриминатора, и насколько хорошо он балансирует с генератором.

Будем считать, что дискриминатор выдает вероятность картинки быть настоящей:

$$D(x) \in [0, 1]$$

Обозначим за p_{data} распределение реальных данных, тогда задачей дискриминатора является максимизировать вероятность реальных данных, и минимизировать вероятность фейковых:

$$\mathbb{E}_{x \sim p_{data}} \left[\log D(x) \right] + \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\log (1 - D(G(z))) \right] \longrightarrow \max_{D(x)}$$

Генератор же наоборот хочет обмануть дискриминатор, а значит максимизировать его вероятность на сгенерированных картинках:

$$\mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\log D(G(z)) \right] \longrightarrow \max_{G(z)} \iff \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\log (1 - D(G(z))) \right] \longrightarrow \min_{G(z)}$$

Так как модели учатся по очереди, получаем итоговую формулу лосса для обучения:

GAN Loss Function

$$\mathcal{L}_{GAN}(G, D) = \min_{G(z)} \max_{D(x)} \left\{ \mathbb{E}_{x \sim p_{data}} \left[\log D(x) \right] + \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\log (1 - D(G(z))) \right] \right\}$$

Существует огромное множество производных моделей для GANов. Можно менять архитектуру модели (StyleGAN v1/v2), а можно придумать более хитрый лосс (LSGAN, WGAN).

1.3. Различные улучшения

Посмотрим на некоторые задачи, которые можно решать при помощи GANов.

Conditional GAN (cGAN)

Мы научились генерировать изображения, но все же хочется как-то контролировать этот процесс. Например, уметь заставить модель сгенерировать картинку с конкретным классом.

Для этого добавим в генератор возможность на вход получать *условие* для генерации, а у дискриминатора сделаем две головы:

$$D_{\varphi}^1(x) \text{ для обычного GAN лосса, } D_{\varphi}^2(x) \text{ для лосса классификации картинок}$$

Таким образом, будем учить дискриминатор не только отличать реальные картинки от фейковых, но и классифицировать изображения по каким-то заранее заданным классам.

Для этого нужно добавить в исходную функцию потерь еще и классификационный лосс, чтобы генератор также получал штраф за картинки неправильного класса.

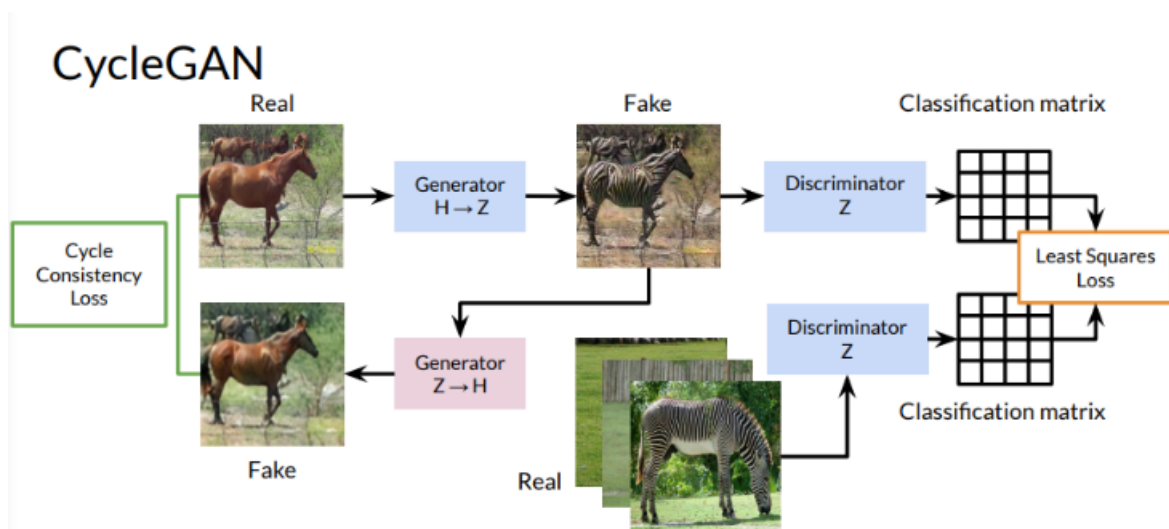
Image2Image задачи

Формулировка задачи: необходимо перевести картинку из одного распределения в картинку из другого распределения, при этом сохранив некое соответствие.

Бывают парные датасеты, где каждой картинке из одного распределения (домена) соответствует конкретная картинка из второго. Например, датасет Sat2Map состоит из пар (изображение местности со спутника, схема с гугл карты).

Задача становится сильно сложнее, когда есть только два датасета картинок, но нет соответствия между ними. Например, у нас есть множество картинок художника и реальные фотографии, на которые мы бы хотели накладывать стиль этого художника.

Для этой задачи хорошо подходит Cycle Consistency Loss, который используется в архитектуре CycleGAN. Идея в том, что для каждого из двух датасетов у нас будет свой GAN, который должен генерировать картинки из другого множества. Циклический консистентный лосс в свою очередь следит за тем, чтобы при прогоне картинки из одного множества в другое, а затем обратно, она не поменялась.



Мы рассмотрим более простой случай **Pix2Pix**, где есть парные данные. Формально, пусть имеется пара соответствующих друг другу картинок из двух распределений:

$$x_s \sim p_s(x), x_t \sim p_t(x) \implies \text{хотим } G(x_s) \approx x_t$$

Для этого можно взять обычный попиксельный MSE Loss:

$$\mathbb{E}_{x_s, x_t} \left[\|G(x_s) - x_t\|_2^2 \right]$$

Однако обучая только на него, картинки будут получаться очень размытыми, так как MSE предполагает нормальность распределения пикселей, что в реально не так.

Поэтому дополнительно добавляется еще и GAN Loss, который вводится именно для повышения реалистичности генерации, а не проверки соответствия пар картинок друг другу.

Объединяя эти два подхода, получаем составной лосс для обучения:

$$\mathcal{L} = \alpha \text{MSE} + (1 - \alpha) \mathcal{L}_{GAN}(G, D) \longrightarrow \min_{G, D}, \quad \alpha \in [0, 1]$$

1.4. Теоретические обоснование

Попробуем понять, что на самом деле происходит при оптимизации GAN лосса. Вспомним, что генератор получает $z \sim \mathcal{N}(0, I)$; для удобства обзовем распределение его выходов как

$$x = G(z) \implies x \sim p_{gen}(x)$$

Зафиксируем генератор, тогда GAN лосс принимает вид:

$$\mathcal{L}_{GAN}(D) = \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{x \sim p_{gen}} [\log (1 - D(x))] \longrightarrow \max_{D(x)}$$

Найдем оптимальный дискриминатор $D^*(x)$:

$$\mathcal{L}_{GAN}(D) = \int_{x \in \mathcal{X}} [p_{data}(x) \log D(x) + p_{gen}(x) \log (1 - D(x))] dx \longrightarrow \max_{D(x)}$$

Как это максимизировать по всему пространству картинок $\mathcal{X}$ не очень понятно. Упростим нашу задачу и будем максимизировать функцию под интегралом в каждой точке:

$$c := D(x) \implies g(c) = p_{data}(x) \log c + p_{gen}(x) \log (1 - c) \longrightarrow \max_c$$

$$g'(c) = \frac{p_{data}(x)}{c} - \frac{p_{gen}(x)}{1 - c} = 0 \implies (1 - c) p_{data}(x) - c p_{gen}(x) = 0$$

Отсюда получаем оптимальный дискриминатор:

$$D^*(x) = c^* = \frac{p_{data}(x)}{p_{data}(x) + p_{gen}(x)}$$

Действительно, если генератор идеально обучился, то дискриминатор не сможет отличать реальные картинки от сгенерированных, а значит

$$p_{data}(x) = p_{gen}(x) \implies D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_{gen}(x)} = \frac{1}{2}$$

Теперь подставим оптимальный дискриминатор в лосс:

$$\mathcal{L}_{GAN}(D^*) = \int_{x \in \mathcal{X}} \left[p_{data}(x) \log \frac{p_{data}(x)}{p_{data}(x) + p_{gen}(x)} + p_{gen}(x) \log \frac{p_{gen}(x)}{p_{data}(x) + p_{gen}(x)} \right] dx = (*)$$

Выражения очень похожие на KL-дивергенцию, но для этого нам нужно отнормировать удвоенную плотность. Прибавим и вычтем под интегралом $p_{data}(x) \log 2$ и $p_{gen}(x) \log 2$:

$$\begin{aligned} (*) &= \int_{x \in \mathcal{X}} \left[p_{data}(x) \log \frac{2 p_{data}(x)}{p_{data}(x) + p_{gen}(x)} + p_{gen}(x) \log \frac{2 p_{gen}(x)}{p_{data}(x) + p_{gen}(x)} \right] dx - \\ &- \log 2 \int_{x \in \mathcal{X}} [p_{data}(x) + p_{gen}(x)] dx = \text{KL} \left(p_{data} \parallel \frac{p_{data} + p_{gen}}{2} \right) + \text{KL} \left(p_{gen} \parallel \frac{p_{data} + p_{gen}}{2} \right) - 2 \log 2 = \\ &= 2 JS(p_{data}, p_{gen}) - 2 \log 2 \end{aligned}$$

Дивергенция Йенсена—Шеннона

$$JS(p \parallel q) = \frac{1}{2} \text{KL}(p \parallel q) + \frac{1}{2} (q \parallel p)$$

Дивергенция Йенсена—Шеннона все еще не является метрикой, так как для нее не выполняется неравенство треугольника. Зато по сравнению с KL она симметрична и ограничена.

Можно сделать вывод, что при обучении GAN минимизируется JS-дивергенция между распределением данных и генератора. На самом деле это не совсем так, потому что в реальности мы никогда не получим идеальный дискриминатор.

Более того, если переобучить дискриминатор, то возникнет проблема затухания градиентов, при которой генератор перестанет учиться. То есть оптимальный генератор с точки зрения практики, наоборот, все ломает.

1.5. Mode collapse

Слабым место GANов является **mode collapse** – свойство модели покрывать лишь очень небольшое количество мод. Иначе говоря, несмотря на то, что GANы генерируют супер реалистичные картинки, разнообразие семплов будет сильно ограничено. Решение этой проблемы не придумали, поэтому если mode collapse критичен, придется использовать другие модели.

